

深度学习知识

1. Softmax 如何防止指数上溢和下溢

解决上溢0

核心思路：减去最大值，让所有指数输入 ≤ 0 ，从根源避免上溢。

原理

原始的Softmax函数：
$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

当 x_i 很大的时， e^{x_i} 会直接上溢 (Inf)。

稳定版 Softmax：
$$\text{softmax}(x_i) = \frac{e^{x_i - M}}{\sum_j e^{x_j - M}}, M = \max(x)$$
，此时 $x_i - M \leq 0$ ，所以 $e^{x_i - M} \leq 1$

因为
$$\frac{e^{x_i - M}}{\sum_j e^{x_j - M}} = \frac{e^{x_i} e^{-M}}{\sum_j e^{x_j} e^{-M}} = \frac{e^{x_i}}{\sum_j e^{x_j}}$$
，结果是等价的。

代码实现

代码块

```
1 import numpy as np
2
3 def softmax(x):
4     # 防止上溢：减去最大值
5     x_shifted = x - np.max(x)
6     exp_x = np.exp(x_shifted)
7     return exp_x / np.sum(exp_x)
```

解决下溢

下溢是指数值过小，小于浮点数能表示的最小值，最终被舍入为 0，可以通过通过数值稳健性处理和工程技巧规避

1. 通过添加极小值eps来防止分母为0

代码块

```
1 import numpy as np
2
```

```

3  def stable_softmax(x, eps=1e-10):
4      """
5      防上溢+防下溢的Softmax实现
6      :param x: 输入向量/矩阵 (shape: [N,] 或 [batch, N])
7      :param eps: 极小值, 防止分母为0
8      :return: Softmax结果
9      """
10     # 步骤1: 减最大值, 杜绝上溢 (核心)
11     # 若为矩阵 (如批量数据), 按行 (每个样本) 取最大值
12     axis = 1 if x.ndim == 2 else 0
13     x_max = np.max(x, axis=axis, keepdims=True)
14     x_shifted = x - x_max
15
16     # 步骤2: 计算指数 (此时所有指数项 ≤ 1, 无场上溢)
17     exp_x = np.exp(x_shifted)
18
19     # 步骤3: 求和并加极小值eps, 防止分母为0 (解决下溢导致的0/0)
20     sum_exp_x = np.sum(exp_x, axis=axis, keepdims=True) + eps
21
22     # 步骤4: 计算Softmax
23     return exp_x / sum_exp_x
24

```

2. LogSoftmax

在分类任务 (如交叉熵损失) 中, 我们通常需要计算 $\log(\text{softmax}(x))$, 直接计算会因 $\text{softmax}(x)$ 极小导致下溢, 因此直接实现 LogSoftmax 更高效

$$\text{logsoftmax}(x_i) = x_i - M - \log\left(\sum_j e^{x_j - M}\right)$$

代码块

```

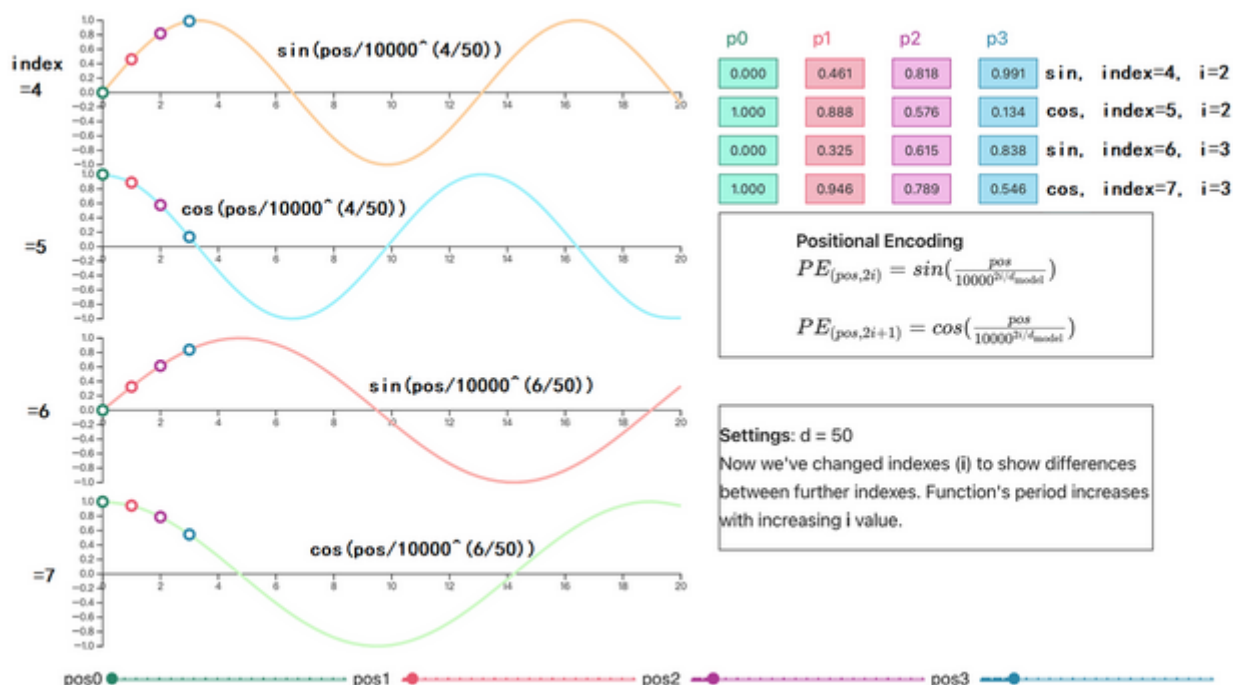
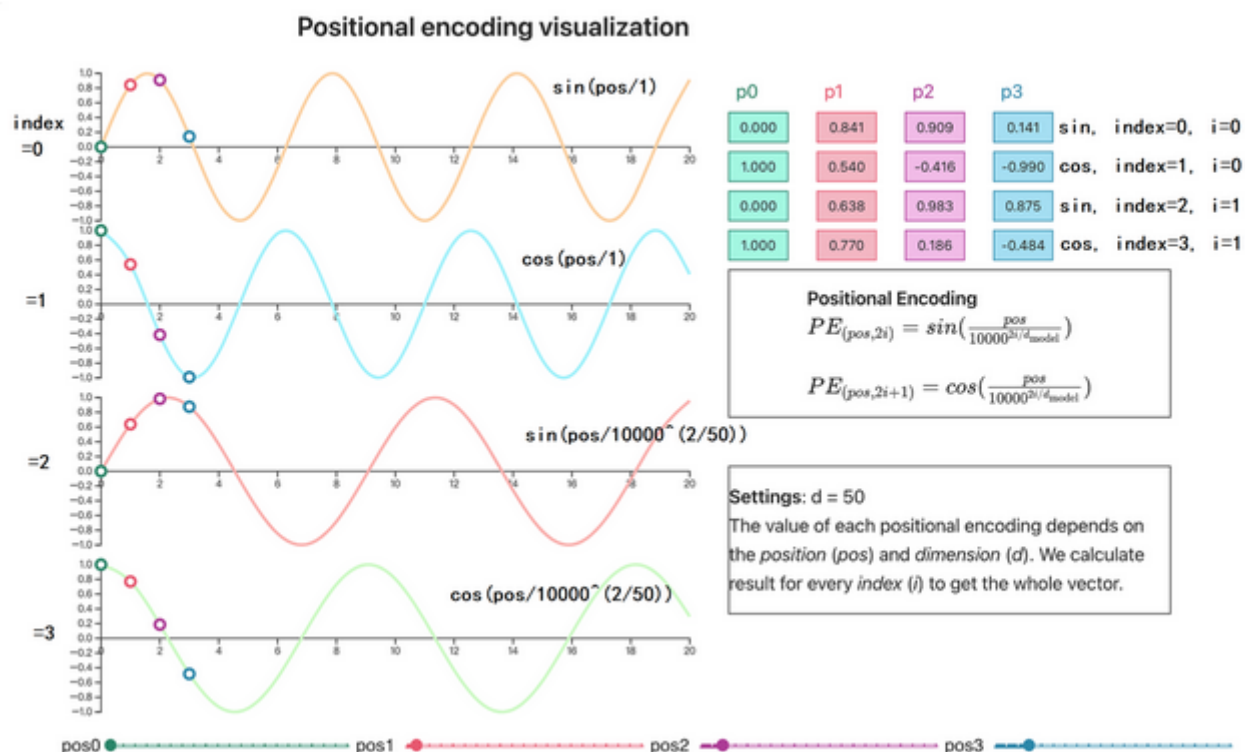
1  def stable_log_softmax(x):
2      axis = 1 if x.ndim == 2 else 0
3      x_max = np.max(x, axis=axis, keepdims=True)
4      x_shifted = x - x_max
5      sum_exp = np.log(np.sum(np.exp(x_shifted), axis=axis, keepdims=True))
6      return x_shifted - sum_exp

```

2. Transformer 中的 Positional Encoding (位置编码)

Transformer 是全注意力结构，没有 CNN 的局部感受野、也没有 RNN 的时序递归，天生不知道词的顺序 / 位置，所以必须人为加入位置信息，这就是 **Positional Encoding**（位置编码）。

A. 索引|index



位置公式: $PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right), PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{model}}}}\right)$

pos : 词在句子里的位置

i : 向量的维度下标

`d_model`：词向量总维度

特点：

1. 不同的位置有不同编码，不同 pos 采样不同，编码不同
2. 不同长度句子也能通用
3. 可以学习到相对位置 $\sin(a + b) = \sin a \cos b + \cos a \sin b$ ，这表示k+offset的编码可以用位置k的编码来进行线性表示

通过数学原理解释Transformer 为什么Transformer能通过正余弦编码学到相对位置

首先有正弦 / 余弦的和角公式：

$$\sin(a + b) = \sin a \cos b + \cos a \sin b, \cos(a + b) = \cos a \cos b - \sin a \sin b$$

此时假设我们有位置 `pos` 的编码： $PE_{pos,2i} = \sin(\theta), PE_{pos,2i+1} = \cos(\theta)$ ($\theta = \frac{pos}{10000^{\frac{2i}{d_{model}}}}$)

偏移量 `k`：我们想表示「pos + k」这个位置（即 pos 后面第 k 个词）

那么位置 `pos + k` 的编码：

$$PE_{pos+k,2i} = \sin(\theta + \frac{k}{10000^{\frac{2i}{d_{model}}}}), PE_{pos+k,2i+1} = \cos(\theta + \frac{k}{10000^{\frac{2i}{d_{model}}}}),$$

令 $\delta = \frac{k}{10000^{\frac{2i}{d_{model}}}}$ ，那么

$$PE_{pos+k,2i} = \sin(\theta) \cos(\delta) + \cos(\theta) \sin(\delta), PE_{pos+k,2i+1} = \cos(\theta) \cos(\delta) - \sin(\theta) \sin(\delta)$$

最终有

$$\begin{aligned} PE_{pos+k,2i} &= PE_{pos,2i} \cos(\delta) + PE_{pos,2i+1} \sin(\delta) \\ PE_{pos+k,2i+1} &= PE_{pos,2i+1} \cos(\delta) - PE_{pos,2i} \sin(\delta) \end{aligned}$$

位置 pos+k 的编码 = 位置 pos 的编码 × 一组固定系数（只和 k 有关）

Transformer 的 Attention 层是线性变换 + 加权求和，完全能学会这个线性关系，从而理解「pos 和 pos+k 之间差了 k 个位置」

基于学习的绝对位置编码

基于学习的绝对位置编码是让模型自己学一组“位置向量”，直接加到词嵌入上。它简单、效果好，但只能表示绝对位置，不能外推到更长序列。

代码块

```
1 import torch
2 import torch.nn as nn
3
4 class LearnedAbsolutePositionalEncoding(nn.Module):
5     def __init__(self, d_model: int, max_len: int = 512, dropout: float = 0.1):
```

```

6         super().__init__()
7         # 向量维度
8         self.d_model = d_model
9         # 句子最大长度
10        self.max_len = max_len
11        self.dropout = nn.Dropout(dropout)
12
13        # 可学习位置嵌入
14        self.pos_emb = nn.Embedding(max_len, d_model)
15
16        def forward(self, x):
17            # x: [batch_size, seq_len, d_model]
18            batch_size, seq_len, _ = x.shape
19
20            # 位置序列
21            pos = torch.arange(seq_len, device=x.device)
22            # 查表得到位置编码
23            pos_encoding = self.pos_emb(pos) # [seq_len, d_model]
24
25            # 加到输入上
26            x = x + pos_encoding
27            return self.dropout(x)

```

旋转位置编码（RoPE - Rotary Position Embedding）

RoPE 不直接给位置加向量，而是把 Q、K 向量在高维空间里 “旋转一个角度”，角度大小由位置决定。

$$Attention(Q, K, V) = softmax \frac{QK^T}{\sqrt{d_k}} V$$

RoPE 做的事：

- 对 **位置 m** 的 query：把 Q_m 旋转 $m \cdot \theta_i$
- 对 **位置 n** 的 key：把 K_n 旋转 $n \cdot \theta_i$

旋转矩阵：

$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$ ，该矩阵的核心性质是**1.正交性 2. 旋转后向量的模长不变 3. 旋转不改变向量间的夹角和内积的相对大小。**

Transformer 的 Q/K 维度 d_k 通常是偶数（如 64、128），因此可以把整个高维向量拆成若干个 2 维子空间：

$$X = [x_1, x_2, x_3, x_4, \dots, x_{d_k-1}, x_{d_k}]$$

每个子空间 (x_{2i-1}, x_{2i}) 用不同频率的 θ_i 旋转，整体旋转矩阵是**分块对角矩阵**：

$$R_{dk}(\theta_1, \theta_2, \dots, \theta_{\frac{d_k}{2}}) = \begin{bmatrix} R(\theta_1) & 0 & \dots & 0 \\ 0 & R(\theta_2) & \dots & 0 \\ \dots & \dots & \dots & \dots \\ 0 & \dots & \dots & R(\theta_{\frac{d_k}{2}}) \end{bmatrix}, \text{分块旋转可以保证每个2维子空间独立旋}$$

转互不干扰，且不同的子空间使用不同的频率 $\theta_i = 10000^{\frac{2i}{d_k}}$ ，覆盖了不同粒度的位置信息。

对于 (x_1, x_2) ，表示为复数 $z = x_1 + ix_2$ ， $e^{i\theta} = \cos \theta + i \sin \theta$ ，他们相乘可以得到

$$(x_1 + ix_2) \cdot (\cos \theta + i \sin \theta) = (x_1 \cos \theta - x_2 \sin \theta) + i(x_1 \sin \theta + x_2 \cos \theta)$$

对于m位置的Q向量和n位置的K向量进行旋转等价于 $q_m' = q_m \cdot e^{im\theta_i}$, $k_n' = k_n \cdot e^{in\theta_i}$

$$\langle q_m', k_n' \rangle = q_m' \cdot \bar{k}_n' = q_m e^{im\theta_i} \cdot \bar{k}_n e^{-in\theta_i} = q_m \bar{k}_n \cdot e^{i(m-n)\theta_i}$$

内积结果只和相对位置差 (m-n) 有关，绝对位置 m/n 被消掉了——这就是 RoPE 能编码相对位置的数学本质！

3. 求似然函数的步骤

定义：概率是给定参数，求某个事件发生概率；似然则是给定已发生的事件，估计参数。

1. 写出单个样本的分布： $f(x; \theta)$

2. 利用独立性写出联合分布： $L(\theta) = \prod_{i=1}^n f(x_i; \theta) = f(x_1; \theta)f(x_2; \theta)\dots f(x_n; \theta)$ ，这就是似然函数

3. 取自然对数，得到对数似然： $\ell(\theta) = \ln L(\theta) = \sum_{i=1}^n \ln f(x_i; \theta)$

4. 对 θ 求导并令导数等于0，解出的 $\hat{\theta}$ 就是极大似然估计。

4. 对比学习

定义：让相似的样本在特征空间中离得更近，让不相关的样本在特征空间中离得更远。

优势：不需要人工标注标签，可以利用**数据本身的结构**做自监督；学到的特征具有**通用性、鲁棒性**，可以迁移到分类、检测、检索、识别等下游任务。

标准流程

1. 对同一张图片进行两次不同的数据增强得到正样本对，batch中的其他图片作为负样本

2. 用编码器 $f(\cdot)$ 分别提取正样本对 x_1, x_2 的特征 $z_1 = f(x_1), z_2 = f(x_2)$

3. 使用对比损失约束，拉近 z_1, z_2 ，使 z_1 远离所有的负样本对

常用的相似度函数

1. 余弦相似度: $\cos(u, v) = \frac{u \cdot v}{\|u\|_2 \|v\|_2}$, 看两个向量的夹角, 输出范围[-1,1], 值越大越相似
2. 欧氏距离: $d(u, v) = \|u - v\|_2$, 距离越小越相似, 用得少
3. 曼哈顿距离: $d(u, v) = \sum_{i=1}^d |u_i - v_i|$, 衡量两个点在各维度上绝对差值的总和, 适合于离散数据, 如推荐系统中, 对稀疏离散特征做用户 / 物品距离度量
4. Jaccard 相似度: 对于两个集合 S_1 和 S_2 , $Jaccard(S_1, S_2) = \frac{|S_1 \cap S_2|}{|S_1 \cup S_2|}$, 针对「集合」的相似度度量, 适合用于文本相似度, 用户行为分析, 图像分割/目标检测和稀疏二值数据。
5. 皮尔逊相关系数: $r(X, Y) = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^n (X_i - \bar{X})^2} \sqrt{\sum_{i=1}^n (Y_i - \bar{Y})^2}}$, 衡量两个连续变量之间线性相关程度的指标, 适合在推荐系统中衡量用户评分的线性相似度。

损失函数

1. Contrastive Loss: $\iota = y \cdot \|f_i - f_j\|^2 + (1 - y) \cdot \max(m - \|f_i - f_j\|, 0)^2$, $y=1$: 正样本对 $\rightarrow$ 直接拉近距离; $y=0$: 负样本对 $\rightarrow$ 只在距离小于 m 时才惩罚, 强制拉到阈值外。
2. 三元组损失: $\iota = \max(\|f_a - f_p\|^2 - \|f_a - f_n\|^2 + m, 0)$, 让锚点到正样本的距离 < 锚点到负样本的距离

两种方法因为使用的负样本数量少所以存在收敛慢, 陷入局部最小值等问题。

3. InfoNCE Loss: $\iota_{InfoNCE} = -\log \frac{\exp(\text{sim}(q, k_+)/\tau)}{\sum_{i=0}^K \exp(\text{sim}(q, k_i)/\tau)}$, q 表示锚点特征, k_+ : 和 q 同源的正样本, k_i : 1个正样本+ K 个负样本, τ : 温度系数, 一般是0.07-0.2。把 “从一堆负样本里找出自己的正样本” 当成一个 **K+1 分类任务**, 让模型分对。
4. SupCon Loss: 在有标签的时候用, 形式和InfoNCE Loss一致, 同一类的所有样本, 都互为正样本, 其余类别为负样本。

对比学习经典模型

1. MoCo v1 / v2 / v3 (何恺明团队, FAIR) : a.使用动量编码器。 B. 维护一个大队列作为负样本库, 不依赖与大Batch
2. SimCLR / SimCLRv2 (Google) :强数据增强 + MLP 投影头 + InfoNCE + 超大 batch;
3. SimSiam (字节跳动) :只使用正样本对, 孪生网络 + stop-gradient (停止梯度) + 预测头;
4. CLIP (OpenAI) : 使用图片编码器 + 文本编码器; 同一张图 + 描述是正样本, 跨图文是负样本;
5. SwAV: 先对特征做聚类, 用聚类伪标签做对比;

5. 常见的损失函数

KL散度

定义：表示相对熵，衡量**真实分布** P 与**模型分布** Q 之间的差异。

公式： $D_{KL}(P||Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$

特点：1. 非负性。2. 不对称性， $D_{KL}(P||Q) \neq D_{KL}(Q||P)$ ，在实际使用中，必须注意哪个分布是“基准真相”，哪个是“预测值”。3. 衡量分布差异

用途：分布匹配、知识蒸馏、VAE

具体使用：VAE中想要模型学习到的潜在变量 z 的分布 $Q(z|x)$ 尽量接近标准正态分布；

公式关系：**交叉熵 = 熵 + KL散度**， $H(P, Q) = H(P) + D_{KL}(P||Q)$ ， $H(P)$ 是固定的常数与模型无关，训练时，最小化交叉熵 = 最小化KL散度

代码：计算两个批量分布之间的 KL 散度

代码块

```
1  import torch
2  import torch.nn.functional as F
3
4  # 假设 batch_size=2, 每个分布有 3 个类别
5  P = torch.tensor([[0.2, 0.5, 0.3],
6                    [0.1, 0.8, 0.1]]) # target: 真实分布
7
8  Q_logits = torch.tensor([[1.0, 2.0, 0.5],
9                            [0.5, 2.5, 0.2]]) # 模型输出的 logits
10
11 # 将 Q_logits 转换为 log-probabilities
12 log_Q = F.log_softmax(Q_logits, dim=-1)
13
14 # 计算 KL(P || Q)
15 kl = F.kl_div(log_Q, P, reduction='batchmean')
16 print("KL divergence (P||Q):", kl.item())
```

交叉熵损失

定义：用于衡量模型预测的概率分布与真实标签分布之间的差异。

公式

- 1. 多分类：** $Loss = - \sum_i y_i \log \hat{y}_i$ ($\hat{y}_i$ 是模型预测概率，可以使用Softmax转化)，如果真实标签分布是 P ，模型预测的概率分布是 Q ，可以写成
$$Loss = H(P, Q) = - \sum_i P(x) \log Q(x) = - \sum_i P(x) \log P(x) + \sum_x P(x) \log \frac{P(x)}{Q(x)} = H(P) + D_{KL}(P||Q)$$
一般情况下真实标签是长度为 C 的 one-hot 向量 y （其中正确类别 c 处为 1，其余为 0），上面的式子可以简化为 $Loss = -\log(\hat{y}_c)$
- 2. 二分类：**是多分类的特例，通常使用Sigmoid 函数输出一个概率值 $\hat{y}$ （表示正类概率），真实标签 $y \in \{0, 1\}$ ，二元交叉熵损失为： $Loss = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$ （可以自己带入数据算一下）

代码

代码块

```
1 import torch
2 import torch.nn as nn
3
4 criterion = nn.CrossEntropyLoss()
5 logits = torch.tensor([[2.0, 1.0, 0.1]]) # 模型输出, 形状 (1, C)
6 labels = torch.tensor([0]) # 真实类别, 形状 (1)
7 loss = criterion(logits, labels) # 计算损失
```

MSE（均方误差）

定义：衡量的是模型预测的值与真实值之间的**差异**

公式：
$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2,$$

y_i ：第 i 个样本的真实值。

$\hat{y}_i$ ：第 i 个样本的预测值。

n ：样本总数。

为什么分类用交叉熵

- 1. MSE 极易梯度消失：**MSE + Sigmoid/Softmax，预测接近正确或者完全错误时梯度几乎为 0，很难收敛；交叉熵在错的时候梯度大，修正快。
- 2. 任务不匹配：**分类是拟合**概率分布**，交叉熵天生衡量分布差异；MSE 是回归损失，适合连续值。
- 3. 惩罚力度：**交叉熵对分错样本惩罚远大于 MSE，更利于学分类边界；
- 4. 交叉熵损失是凸优化**，MSE 在分类中是非凸，易局部最优。

多任务学习 loss 差异过大的解决方法

不同任务 loss 量级差太多，会导致模型只学大 loss 任务。

1. 人工加权
2. 对Loss进行归一化
3. 添加自适应权重，让模型自动学习。
4. 进行梯度归一化
5. 先训练主任务，再微调

如何解决Softmax指数上溢的问题

[国深度学习知识](#)

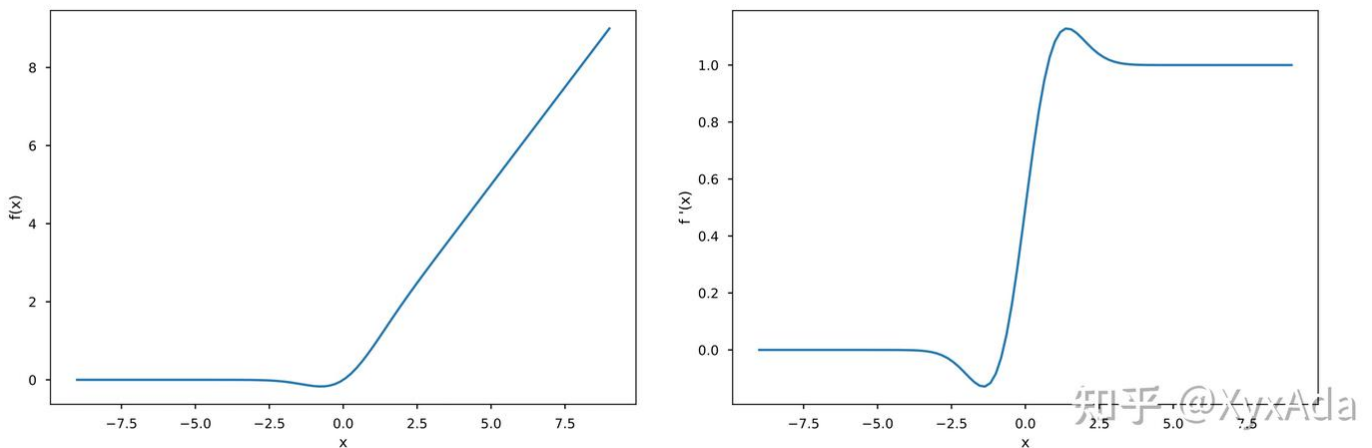
6. 激活函数

1. GELU (Gaussian Error Linear Unit)

定义：高斯误差线性单元，是 ReLU 的平滑变体，通过高斯分布累积函数对输入进行自适应加权，兼顾 ReLU 的稀疏性与平滑梯度特性。

公式： $GELU(x) \approx 0.5x \cdot (1 + \tanh(\sqrt{2/\pi} \cdot (x + 0.044715x^3)))$

GELU function and it's Derivative



优势

1. **平滑性：**负区间非零输出，梯度连续，训练更稳定
2. **自适应性：**输入越大权重越接近 1，越小越接近 0，类似自门控
3. **稀疏激活：**多数负输入被抑制，保留 ReLU 优势
4. **梯度更优：**避免 ReLU 的“死亡神经元”问题

2. Swish

定义：自门控 Sigmoid 激活函数，引入可学习参数 β ，自适应调整门控强度，由 Google Brain 于 2017 年提出。

公式： $Swish(x) = x \cdot sigmoid(\beta x)$ ，当 $\beta=1$ 时为简化版；当 $\beta \rightarrow \infty$ 时趋近 ReLU； $\beta=0$ 时退化为线性函数。

特点

1. **自门控机制：**输入自身控制输出权重，增强表达能力
2. **非线性更强：**相比 GELU，对输入变化更敏感
3. **参数化：** β 可学习，适配不同任务与数据分布
4. **平滑梯度：**负区间有非零输出，梯度更稳定

3. FFN块（Feed-Forward Network）

定义：FFN 块是 Transformer 架构的核心组件，位于自注意力层之后，对每个位置的特征向量进行独立的非线性变换，提供模型的非线性表达能力，占 Transformer 总参数约 **2/3**。

公式： $FFN(x) = f(xW_1 + b_1)W_2 + b_2$

f 为激活函数（早期常用 ReLU，现代大模型多用 GELU、Swish 或门控变体）

W_1 、 b_1 ：升维变换参数（如从 $d_{model}=512 \rightarrow d_{ff}=2048$ ）

W_2 、 b_2 ：降维变换参数（如从 $d_{ff}=2048 \rightarrow d_{model}=512$ ）

特点

1. **逐位置独立处理：**对序列中每个 token 单独加工，不跨位置交互
2. **升维 - 非线性 - 降维：**扩展特征空间 $\rightarrow$ 引入非线性 $\rightarrow$ 还原维度
3. **残差连接 + 层归一化：**标准结构为 “注意力 $\rightarrow$ Add&Norm $\rightarrow$ FFN $\rightarrow$ Add&Norm”
4. **参数密集：**决定模型容量，是性能提升的关键模块

应用场景：Transformer 编码器 / 解码器的核心层，被 BERT、GPT、T5 等所有主流大模型采用。

4. GLU（Gated Linear Unit）线性门控单元

定义：门控线性单元，通过门控机制选择性传递信息，结合 LSTM 门控思想与 MLP 结构，由两个并行分支（值分支 + 门控分支）组成。

公式： $GLU(x) = (xW_1 + b_1) \odot \sigma(xW_2 + b_2)$ ，

- ⊙：哈达玛积（元素级乘法）
- σ：sigmoid 激活函数（门控值∈[0,1]）
- W₁、b₁：值分支线性变换
- W₂、b₂：门控分支线性变换

变体

变体	公式	特点	应用
SwiGLU	Swish_β(xW + b) ⊙ (xV + c)	门控用 Swish 替代 sigmoid	PaLM、LLaMA 2、GPT-NeoX
GeGLU	GELU(xW + b) ⊙ (xV + c)	门控用 GELU 替代 sigmoid	GLU 的平滑改进版
FFN→GLU	GELU(xW ₁)W ₂ → SwiGLU(xW, xV)	双线性投影 + 门控，参数效率更高	现代大模型主流替代方案

Transformer 架构中传统 FFN 块被 GLU（门控线性单元）系列变体替代

7. 注意力（Attention）

定义：注意力机制的核心思想是模拟人类的选择性注意力机制，让模型在处理信息时，动态聚焦于关键输入内容，抑制无关噪声，实现资源的高效分配。

1. 加性注意力（多层感知机注意力）

核心思想：将查询向量 *q* 和键向量 *k* 拼接（或相加）后，喂入一个简单的单层前馈神经网络中，通过训练该网络来学习并输出一个注意力分数。

公式： $Score(q, k) = v^T tanh(W_qq + W_kk)$

- W_q：查询的权重矩阵（维度通常是 *d_h* × *d_q*）。
- W_k：键的权重矩阵（维度通常是 *d_h* × *d_k*）。
- v*：是一个权重向量（维度通常是 *d_h* × 1）。
- tanh：激活函数，引入非线性。
- d_h*：注意力隐藏层的维度（超参数）。

特点：比点积注意力更通用（不要求维度一致）但计算开销更大。在处理维度较高或查询与键维度不一致时，理论上更灵活

2. 乘性注意力（点积注意力）

核心思想：通过计算查询向量 *q* 和键向量 *k* 之间的点积（内积）来衡量它们的相似度。

如果两个向量的方向一致（夹角小），点积结果就大，意味着注意力分数高；如果方向垂直或相反，点积结果小或为负，意味着注意力分数低。

公式: $score(q, k) = \frac{q^T k}{\sqrt{d_k}}$

$\sqrt{d_k}$: 缩放因子, d_k 是键向量的维度

实际使用的公式: $Attention(Q, K, V) = softmax \frac{QK^T}{\sqrt{d_k}} V$

特点: 计算效率高, 空间利用率高, 但是有维度限制和梯度消失的风险

3. 自注意力

自注意力 (Self-Attention), 也称为**内部注意力** (Intra-Attention), 是一种特殊的注意力机制, 它允许序列中的每个元素与**同一序列中的所有其他元素**计算注意力权重, 从而捕捉序列内部的依赖关系。

核心思想

传统的注意力机制通常作用于**两个不同的序列** (例如, 在机器翻译中, 解码器关注编码器的输出)。而自注意力作用于**单个序列内部**。

过程

1. 输入序列 X 经过嵌入模型得到矩阵 $X \in R^{n \times d}$ 通过三个可训练的权重矩阵 W^Q, W^K, W^V 把输入 X 映射为查询矩阵 Q , 键矩阵 K 和值矩阵 V

$Q = XW^Q, K = XW^K, V = XW^V$, 其中 $Q, K, V \in R^{n \times d_k}$, (一般的话 $d_k = d$ 或者更小。)

2. 通过公式计算注意力分数, 具体公式可以看 [📖 深度学习知识](#)

优势

1. **全局感受野**: 一步到位捕获**任意距离**的依赖关系
2. **并行计算**: 可一次性计算所有位置的注意力, 无时间步依赖, 训练速度快
3. **动态权重**: 权重根据输入内容动态生成 (而非固定卷积核), 更具灵活性
4. **长距离依赖**: 理论上可完美建模长距离关系, 不受梯度消失/爆炸影响

4. 多头自注意力

通过并行执行多个不同的注意力机制, 使模型能够从不同表示子空间捕获输入序列中**多样化的依赖关系**。

例子：“那只猫因为太胖所以跳不上椅子”时，单头注意力可能需要同时关注，语法关系（“猫”与“那只”）；因果关系（“猫”与“太胖”）；动作关系（“猫”与“跳不上”）。

核心思想

多头注意力将查询、键、值分别进行 h 次不同的线性变换，得到 h 组不同的查询、键、值矩阵，然后在每个变换后的空间上独立地执行缩放点积注意力，最后将 h 个头的输出拼接并通过一个线性层融合。

计算步骤

假设输入的嵌入矩阵为 $X \in R^{n \times d_{model}}$ ， n 是序列长度，一共有 h 个头， $d_k = d_{model}/h$ ， d_{model} 是隐藏层的维度。

1. 对于每个头 $i(i = 1, \dots, h)$ ，使用三个可训练的权重矩阵：

a. $W_i^Q \in R^{d_{model} \times d_k}$

b. $W_i^K \in R^{d_{model} \times d_k}$

c. $W_i^V \in R^{d_{model} \times d_k}$

计算该头的查询、键、值： $Q_i = XW_i^Q, K_i = XW_i^K, V_i = XW_i^V$ ； $Q_i, K_i, V_i \in R^{n \times d_k}$

2. 每个头独立计算注意力，

$$head_i = Attention(Q_i, K_i, V_i) = softmax \frac{Q_i K_i^T}{\sqrt{d_k}} V_i, head_i \in R^{n \times d_k}$$

3. 拼接所有头， $Concat(head_1, \dots, head_h) \in R^{n \times (h \cdot d_k)} = R^{n \times d_{model}}$

4. 通过可训练的权重矩阵 $W^O \in R^{d_{model} \times d_{model}}$ 进行线性变换得到最终输出。

作用

1. 每个头可以学习关注不同类型的依赖关系
2. **增强表达能力**：多头机制相当于集成了多个“专家”，每个专家从不同角度分析序列，综合后得到更全面的表示。
3. **稳定训练**：多头允许模型在多个子空间中并行提取特征，避免了单头注意力可能陷入的局部最优或梯度饱和问题

8. YOLO目标检测

任务要求：目标定位 + 目标分类

传统方法（双阶段检测器）：先进行区域候选，在进行边界框预测和分类

YOLO（单阶段检测器）：省略了区域候选步骤，直接在整张图像上进行检测。